High Level Design Document

Introduction

This High Level Design (HLD) document outlines the architecture and core components for **AgentX - Multi-Agent Collaboration Framework**. The project aims to implement a multi-agent system using LangGraph, Langchain, and FastAPI, enabling specialized AI agents to collaboratively solve complex legal domain tasks through dynamic, context-aware workflows.

---

1. System Architecture Overview

Architecture Description:
AgentX is structured as a modular, service-oriented system. It exposes a FastAPI-based REST API, orchestrates agent collaboration via LangGraph, leverages Langchain for agent logic and memory, and integrates with OpenAI for generative AI capabilities.

Module/Component	Role
API Gateway (FastAPI)	Exposes REST endpoints for client interaction and task submission
Orchestrator (LangGraph)	Manages agent workflows, task decomposition, and agent coordination
Agent Layer (Langchain)	Implements specialized legal agents with memory and reasoning
Memory Store	Persists agent context and conversation history
LLM Integration (OpenAI)	Provides generative AI capabilities for agent reasoning and output

---

2. Component Interactions

Sequence Step	Interaction Description
1	Client submits a legal task via FastAPI endpoint
2	Orchestrator decomposes the task and assigns subtasks to specialized agents
3	Agents process subtasks, leveraging memory and LLM, and return results to orchestrator
4	Orchestrator aggregates agent outputs into a final response
5	API Gateway returns the response to the client

---

3. Data Flow Overview

Data Flow	Source	Destination	Description
Task Submission	Client	API Gateway	User submits legal task request
Task Orchestration	API Gateway	Orchestrator	Task is routed for decomposition and assignment

Subtask Processing	Orchestrator	Agents	Agents receive, process, and respond to subtasks
Memory Access/Update	Agents	Memory Store	Agents read/write context and history
LLM Query	Agents	OpenAI API	Agents invoke LLM for reasoning/generation
Aggregated Response	Orchestrator	API Gateway	Final result is returned to the client

---

4. Technology Stack

Technology	Purpose
Python	Core programming language
FastAPI	REST API framework
Langchain	Agent logic, memory, and tool integration
LangGraph	Agent orchestration and workflow engine
OpenAI API	Generative AI (LLM) capabilities

---

5. Scalability & Reliability

- **Scalability:**
  - Stateless API and orchestrator components support horizontal scaling.
  - Agent processes can be distributed or containerized for parallel execution.
  - Memory store can be backed by scalable databases (e.g., Redis, PostgreSQL).
- **Reliability & Security:**
  - API endpoints secured via authentication and rate limiting.
  - Error handling and retry logic for agent and LLM interactions.
  - Logging and monitoring for workflow traceability and debugging.

End of Document